

# WebAuthn PRF-Based Vault Key Wrapping and Zero-Knowledge PWA Storage Architecture

Ian Pinto<sup>1</sup>

Independent Researcher, Mumbai, India  
ianpinto1980@gmail.com

**Abstract.** This paper presents a client-side security architecture that combines the Web Authentication (WebAuthn) pseudo-random function (PRF) extension with symmetric cryptography and carefully engineered browser storage patterns to achieve strong zero-knowledge properties for web-based credential vaults. The design is grounded in the TrustVault-PWA project and informs the reusable `webauthn-prf-zktv` library.

**Keywords:** WebAuthn, PRF, passkeys, zero-knowledge, IndexedDB, PWA, cryptography

## 1 Introduction

The Web Authentication (WebAuthn) standard and FIDO2 ecosystem provide strong public-key based authentication and passkeys that are increasingly used to replace passwords on the web. Recent extensions such as the pseudo-random function (PRF) allow a relying party to request a high-entropy symmetric secret derived from a passkey credential during an authentication ceremony, enabling end-to-end encryption of user data.[9,2]

Password managers and credential vaults seek to offer zero-knowledge security: server-side or client-side storage alone should never be sufficient to reveal user secrets. Achieving this property in a web progressive web application (PWA) requires careful coordination of cryptography, browser APIs, and local storage. This paper formalizes a PRF-based vault key wrapping scheme and an IndexedDB storage architecture designed toward this goal.

### 1.1 Contributions

The main contributions are:

- A formal description of a WebAuthn PRF-based vault key wrapping scheme suitable for PWAs.
- An IndexedDB schema and migration strategy that reinforce zero-knowledge properties.
- A reusable, open-source implementation, the `webauthn-prf-zktv` library, that packages these patterns.[7]
- Discussion of post-quantum considerations and migration paths.

## 2 Background and Related Work

### 2.1 WebAuthn, Passkeys, and PRF

WebAuthn [8] is the W3C standard, developed jointly with the FIDO Alliance’s CTAP2 protocol, for challenge–response authentication with asymmetric credentials held by an *authenticator* (a platform biometric sensor, a security key, or a synced passkey provider). *Passkeys* are discoverable WebAuthn credentials, typically synchronized across a user’s devices by the platform vendor. The `prf` client extension [8,9] exposes the CTAP2 `hmac-secret` capability to web applications: during a ceremony the authenticator evaluates a pseudo-random function (HMAC-SHA-256 over a credential-scoped key) at one or two caller-chosen 32-byte salts, and returns 32-byte outputs. Because evaluation requires the physical authenticator and (with `userVerification:required`) a user-verification gesture, the PRF output is a high-entropy symmetric secret that exists only transiently in client memory — a natural root for client-side encryption keys. Developer-facing treatments of the extension are given by Yubico [9] and the SimpleWebAuthn project [4].

### 2.2 PRF-Based Vault Encryption in Existing Systems

Bitwarden uses the PRF extension to unlock its vault without the master password: the PRF output decrypts a wrapped copy of the account encryption key [2]. The scheme in this paper differs in three ways: (i) the wrap key is derived through HKDF with an explicit versioned domain-separation label rather than used directly; (ii) client-side replay verification (challenge, origin, signature counter) is an integral step of the unlock algorithm; and (iii) the record format and storage schema are specified together, so that the zero-knowledge property can be argued about the *entire* persisted state. Generic PRF helper libraries and file-encryption demonstrations [1] integrate the extension but do not prescribe a complete vault architecture with fallback, migration, and storage semantics.

### 2.3 IndexedDB and Client-Side Storage

IndexedDB is the standard transactional object store available to web applications and service workers, and the only browser storage suitable for structured encrypted records of meaningful size. Data in IndexedDB is protected by the same-origin policy but is plaintext-at-rest from the application’s perspective: any principal that can read the origin’s storage (local malware, forensic imaging, a malicious extension with storage access) reads the stored bytes. The TrustVault-PWA project [6], from which this work is extracted, stores both secrets and their metadata encrypted in Dexie-backed IndexedDB and documents its residual plaintext surfaces, such as breach-detection (HIBP) hash prefixes retained for offline checking. The library presented here inherits that design posture: the storage layer never persists anything from which a key could be recomputed.

### 3 Threat Model

We consider four attacker capabilities:

- **Stored-data compromise.** The attacker obtains a full copy of persisted state — an IndexedDB dump, a device backup, or a synchronized replica — but does not control a live session.
- **Browser compromise.** The attacker executes script in the origin (XSS, malicious dependency) while the user is present.
- **Authenticator compromise.** The attacker gains use of the passkey (stolen unlocked device, cloned credential).
- **Harvest-now, decrypt-later.** The attacker records ciphertexts today in the expectation of future cryptanalytic (including quantum) capability.

The corresponding security goals are: *zero-knowledge storage* — stored-data compromise alone yields no secrets (Sect. 4.3); *bounded exposure* under browser compromise — a script obtains at most the secrets unlocked during its window of control, since wrap keys are non-extractable and PRF outputs are zeroized after use; *cloned-credential detection* via signature-counter verification; and *PQ-adequate* symmetric primitives at the storage layer (Sect. 7). Authenticator compromise combined with user verification defeats any client-side scheme and is out of scope.

## 4 PRF-Based Vault Key Wrapping

### 4.1 High-Level Design

Enrollment (`enrollVault`) and unlock (`unlockVault`) use the 32-byte WebAuthn PRF output as HKDF input keying material to derive a non-extractable AES-256-GCM *wrap key*  $K_w$ , which encrypts the 256-bit vault key  $K_v$ . The only persisted artifact is a `WrappedSecretRecord`:

$$R = (\text{scheme}, \text{ciphertext}, \text{nonce}, \text{salt}, [\text{kdfParams}])$$

where  $\text{scheme} \in \{\text{prf-v1}, \text{pw-v1}\}$ , the nonce is a fresh 96-bit random AES-GCM IV, and the salt is the PRF salt (`prf-v1`) or the scrypt salt (`pw-v1`). `kdfParams` is present iff  $\text{scheme} = \text{pw-v1}$ ; the parser (`parseRecord`) rejects `prf-v1` records carrying it. PRF outputs and wrap keys are never stored: wrap keys are created with `extractable: false` in Web Crypto, and transient PRF-output buffers are zeroized (`.fill(0)`) in `finally` blocks.

HKDF uses SHA-256 with an *empty* salt (valid per RFC 5869 [3], treated as zeros); per-credential domain separation comes from the unique PRF salt, which yields unique input keying material. The HKDF `info` label is the frozen constant

$$\ell_{v1} = \text{"webauthn-prf-zktv vault key wrap v1"}.$$

A legacy label, `"TrustVault Vault Key Wrapping v1"`, survives only inside `fromTrustVaultRecord`, the one-shot migration adapter: it unwraps a

TrustVault-PWA record under the legacy label and re-wraps the same vault key under  $\ell_{v1}$  using the same PRF output, so migration needs no additional ceremony. Both labels are frozen: changing either would silently invalidate every record wrapped under it, so format evolution happens by introducing a new scheme identifier and label (e.g. a future `prf-v2`), never by mutating `v1`.

A password fallback scheme `pw-v1` covers clients where PRF is not viable (notably Android WebView, where PRF does not reach the Credential Manager): the wrap key is derived with memory-hard scrypt ( $N=131072$ ,  $r=8$ ,  $p=1$  as a security floor) instead of HKDF-from-PRF; all other steps are identical.

## 4.2 Pseudo-Code

Algorithm 1 reflects `wrapSecret` in `src/core/wrap.ts` composed with the PRF ceremony (`enrollVault` in `src/webauthn/vault.ts`):

---

**Algorithm 1** Vault key wrapping (`enrollVault`  $\rightarrow$  `wrapSecret`, scheme `prf-v1`)

---

**Require:** WebAuthn credential  $C$  with PRF enabled, PRF salt  $s$  (32 random bytes), vault key  $K_v$  (32 bytes)

**Ensure:** Stored record  $R$  that does not reveal  $K_v$  without  $C$

- 1:  $prf \leftarrow \text{EvaluatePRF}(C, s) \triangleright 32$  bytes; `PrfResultMissingError` if absent or wrong length
  - 2:  $K_w \leftarrow \text{HKDF-SHA256}(\text{ikm}=prf, \text{salt}=\varepsilon, \text{info}=\ell_{v1})$
  - 3: import  $K_w$  as non-extractable AES-256-GCM `CryptoKey`
  - 4:  $\text{nonce} \leftarrow \text{Random}(12)$
  - 5:  $\text{ciphertext} \leftarrow \text{AES-256-GCM-Encrypt}(K_w, \text{nonce}, K_v)$   $\triangleright$  auth tag included
  - 6: zeroize  $prf$   $\triangleright$  **finally** block;  $K_w$  is non-extractable, never in JS memory
  - 7:  $R \leftarrow (\text{scheme}=\text{prf-v1}, \text{ciphertext}, \text{nonce}, \text{salt}=s)$
  - 8: **return**  $R$
- 

A deliberate design point concerns zeroization scope: `wrapSecret` does *not* zeroize the caller’s secret buffer. Ownership of  $K_v$  remains with the caller, which may legitimately need the raw key again — most commonly to wrap the same vault key a second time under `pw-v1` so that a password fallback record can sit alongside the PRF record. Zeroization inside the library is therefore scoped to the buffers the library itself creates: PRF outputs after key derivation, and the transient plaintext produced during unwrapping.

Algorithm 2 reflects `unlockVault`  $\rightarrow$  `unwrapSecret`; note the replay verification step absent from generic treatments:

Failure paths are typed. A wrong key or tampered ciphertext raises `DecryptError`, which is intentionally generic and propagates no cause, so an observer cannot distinguish a wrong-key attempt from corrupted data. Malformed or scheme-mismatched records raise `RecordFormatError`, and challenge, origin, or counter anomalies raise `ReplayError`.

**Algorithm 2** Vault key unwrapping (`unlockVault`  $\rightarrow$  `unwrapSecret`)

**Require:** Credential id, stored record  $R = (\text{prf-v1}, \text{ciphertext}, \text{nonce}, s)$ , last stored signature counter  $c$

**Ensure:** Non-extractable session `CryptoKey`  $K_v$ , new counter  $c'$

- 1:  $(prf, c') \leftarrow \text{EvaluatePRF}(C, s)$   $\triangleright$  assertion ceremony with fresh 32-byte challenge
- 2: verify `clientData`: `type=webauthn.get`, challenge and origin match; verify  $c' > c$  (unless  $c'=0$ ); else throw `ReplayError`
- 3:  $K_w \leftarrow \text{HKDF-SHA256}(\text{ikm}=prf, \text{salt}=\varepsilon, \text{info}=\ell_{v1})$
- 4:  $b \leftarrow \text{AES-256-GCM-Decrypt}(K_w, \text{nonce}, \text{ciphertext})$ ; on any failure throw `DecryptError`  $\triangleright$  never distinguishes wrong key from corrupt data
- 5: reject unless  $|b| = 32$ ; import  $b$  as non-extractable AES-256-GCM `CryptoKey`  $K_v$
- 6: zeroize  $b$  and  $prf$   $\triangleright$  both in `finally` blocks
- 7: **return**  $(K_v, c')$   $\triangleright$  caller persists  $c'$  for the next unlock

### 4.3 Zero-Knowledge Argument

The claim is that the complete persisted state — the record  $R$  and any vault ciphertexts encrypted under  $K_v$  — is computationally useless without a ceremony on the enrolled authenticator. The argument rests on three standard assumptions: the authenticator’s `hmac-secret` PRF is pseudo-random to parties without the credential’s internal key; HKDF-SHA256 is a secure key-derivation function [3]; and AES-256-GCM provides IND-CPA confidentiality and INT-CTXT integrity.

For `prf-v1` records,  $R$  contains only public values: the scheme tag, the AES-GCM ciphertext and nonce, and the PRF salt  $s$ . The salt is an *input* to the PRF, not a secret; by PRF security, the output at  $s$  is indistinguishable from random to anyone who cannot query the authenticator. Hence the wrap key  $K_w = \text{HKDF}(prf, \varepsilon, \ell_{v1})$  is indistinguishable from a uniformly random AES-256 key, and by IND-CPA security the ciphertext reveals nothing about  $K_v$ . No other function of the PRF output or of  $K_v$  is ever persisted (Sect. 5), so stored-data compromise reduces to attacking AES-256-GCM with an unknown random key.

For `pw-v1` records the guarantee is necessarily weaker: the record is exactly as strong as the password under offline guessing, which is why the script cost parameters stored in `kdfParams` are treated as a floor, not a tunable. The domain-separation label  $\ell_{v1}$  ensures that even if the same PRF output were ever used with another HKDF-based scheme, the derived keys would be independent.

## 5 IndexedDB Storage Architecture

### 5.1 Schema Overview

The library’s `ZktvDb` (raw IndexedDB with no wrapper dependency; database name `zktv`, version 1) creates three object stores:<sup>1</sup>

<sup>1</sup> The TrustVault-PWA *application* persists further stores at its own layer — notably breach-detection (HIBP) hash prefixes and ephemeral session metadata [6]. Those are

- **vaults**: composite primary key `[vaultId, scheme]`. Each row holds `vaultId`, `scheme`, the JSON-serialized `WrappedSecretRecord` (version-tagged `v:1`, binary fields base64url-encoded), and `updatedAt`. The composite key lets one vault carry a `prf-v1` record and a `pw-v1` fallback record side by side; `loadWrappedVault` tries `prf-v1` first.
- **credentials**: primary key `credentialId` (base64url), with a secondary index on `vaultId`. Rows hold non-secret metadata only: the owning vault id, the last verified signature counter, the base64url PRF salt, the authenticator transports, and a creation timestamp.
- **meta**: key–value store (`keyPath`: `key`) reserved for application metadata and future migrations.

Every stored row round-trips through `serializeRecord/parseRecord`, so structural invariants (12-byte nonce,  $\geq 16$ -byte salt, ciphertext longer than the 16-byte GCM tag, `kdfParams` present iff `pw-v1`) are re-validated on every load and violations surface as `RecordFormatError`.

## 5.2 Schema Diagram

Figure 1 depicts the three stores and their relationship.

## 5.3 Migrations and Security Properties

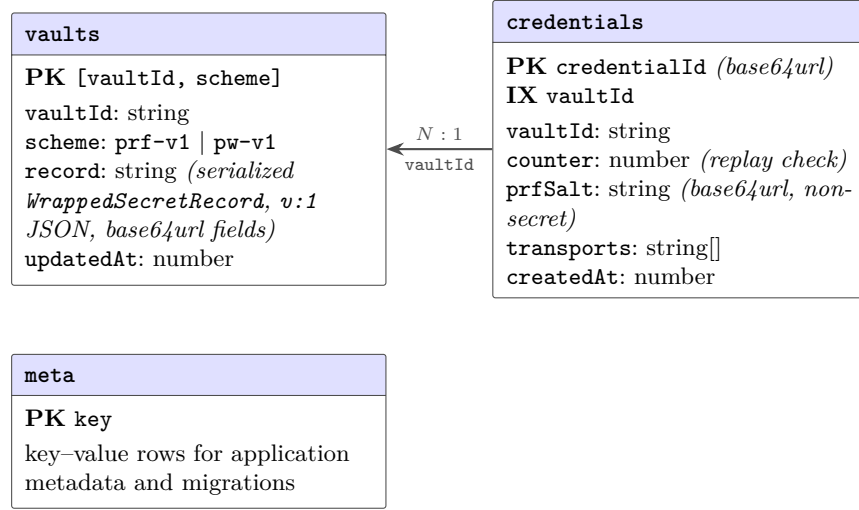
`openVaultDb` accepts an `onUpgrade(db, oldVersion, tx)` hook that runs inside `onupgradeneeded` *after* the built-in stores are ensured, letting applications version their own stores without touching the core schema. The migration invariant is that no upgrade may introduce stored recomputable key inputs: only `WrappedSecretRecord` shapes and non-secret credential metadata are ever persisted. Lifecycle operations preserve this: `clearVault` deletes both scheme rows for a vault and every credential row bound to it via the `vaultId` index, and `securityWipe` clears all three stores (mirroring TrustVault’s security-wipe semantics). Key hygiene throughout: all derived and imported keys are non-extractable `CryptoKeys`, transient buffers are zeroized in `finally` blocks, and the signature counter persisted in `credentials` is updated after each verified assertion (`updateCounter`) to feed the replay check of Algorithm 2.

## 6 Implementation in webauthn-prf-zktv

The library [7] is strict-mode TypeScript, ESM-only, with `@noble/hashes` (scrypt) as its sole runtime dependency; all other cryptography uses the Web Crypto API. It ships three sub-path entry points:

---

application concerns outside the library schema described here, which ships exactly the three stores listed.



**Fig.1.** IndexedDB schema created by openVaultDb (database `zktv`, version 1; `src/indexeddb/db.ts`). One vault may hold up to two wrapped records (`prf-v1` and the `pw-v1` fallback) via the composite primary key, and any number of PRF credentials via the `vaultId` index. Only wrapped records and non-secret credential metadata are ever persisted.

- **webauthn-prf-zktv** (core; Node  $\geq 20$  and browsers): the wrap and unwrap functions of Sect. 4, HKDF and script derivation, record (de)serialization with structural validation, the TrustVault migration adapter, and a typed error hierarchy (`ZktvError`) whose codes distinguish PRF unavailability, cancelled ceremonies, missing PRF results, replay anomalies, decryption failure, record-format violations, and storage errors.
- **webauthn-prf-zktv/webauthn** (browsers only): capability detection and ceremonies. Detection is deliberately non-invasive. `detectPrfSupport` is tri-state (supported, unsupported, unknown) via `getClientCapabilities()` and never creates throwaway credentials to probe; the stricter viability check, `isPrfViableOnThisClient`, additionally rejects Android WebView — where PRF does not reach the Credential Manager — and requires a user-verifying platform authenticator. Enrollment is *adaptive*: `enrollPrfCredential` requests `prf.eval` at creation time, so clients that evaluate PRF during registration (e.g. recent Chrome with Windows Hello) enroll in a single ceremony; otherwise it hard-verifies `prf.enabled` in the extension outputs, aborting cleanly if absent, and runs one assertion ceremony to obtain the PRF output. `evaluatePrf` enforces the 32-byte PRF result length and performs the replay verification of Algorithm 2; `enrollVault` and `unlockVault` compose ceremony, derivation, and (un)wrapping into single calls that zeroize the PRF output in `finally` blocks.

- `webauthn-prf-zktv/indexeddb` (browsers only): the `ZktvDb` storage layer of Sect. 5 (Fig. 1).

The public API is exercised by unit tests covering the failure paths of Sect. 4 (wrong key, malformed records, replayed assertions) alongside the success paths. The core wrap entry point, verbatim from `src/core/wrap.ts`:

**Listing 1.1.** `wrapSecret`: the three wrap paths (PRF, password, caller-supplied wrap key).

```
export async function wrapSecret(options: WrapOptions): Promise<
  WrappedSecretRecord> {
  if ('prfOutput' in options) {
    const wrapKey = await deriveWrapKeyFromPrf(options.prfOutput);
    return encryptRecord(wrapKey, 'prf-v1', options.prfSalt, options.
      secret);
  }
  if ('password' in options) {
    const salt = options.salt ?? generateSalt();
    const kdfParams = options.kdfParams ?? DEFAULT_SCRIPT_PARAMS;
    const wrapKey = await deriveWrapKeyFromPassword(options.password,
      salt, kdfParams);
    const record = await encryptRecord(wrapKey, 'pw-v1', salt, options.
      secret);
    return { ...record, kdfParams };
  }
  return encryptRecord(options.wrapKey, options.scheme, options.salt,
    options.secret);
}
```

**Listing 1.2.** HKDF derivation with the frozen v1 domain-separation label (`src/core/derive.ts`).

```
export const HKDF_INFO_V1 = new TextEncoder().encode(
  'webauthn-prf-zktv_vault_key_wrap_v1',
);

export async function deriveWrapKeyFromPrf(prfOutput, info = HKDF_INFO_V1) {
  {
    const ikm = await crypto.subtle.importKey('raw', prfOutput, 'HKDF',
      false, ['deriveKey']);
    return crypto.subtle.deriveKey(
      { name: 'HKDF', hash: 'SHA-256', salt: new Uint8Array(0), info },
      ikm,
      { name: 'AES-GCM', length: 256 },
      false, // non-extractable
      ['encrypt', 'decrypt'],
    );
  }
}
```



## 7 Post-Quantum Considerations

The storage layer of this architecture is built entirely from symmetric primitives — HMAC-SHA-256 (inside the authenticator PRF), HKDF-SHA256, scrypt, and AES-256-GCM. Against a quantum adversary, Grover-type speedups at most halve the effective security exponent, leaving AES-256 and SHA-256-based constructions with comfortable margins; NIST’s transition guidance accordingly treats AES-256 and SHA-256/384 as quantum-adequate [5]. Harvest-now, decrypt-later attacks against a captured IndexedDB dump are therefore not materially assisted by future quantum capability.

The asymmetric layer is different: today’s passkeys sign with elliptic-curve or RSA algorithms (the library requests ES256 and RS256), which fall to a cryptographically relevant quantum computer. However, that layer protects *authentication*, not the confidentiality of stored records: breaking the signature scheme lets an attacker forge assertions toward a relying party, but does not reveal the `hmac-secret` key inside the authenticator, and hence not the PRF output or  $K_v$ . Because the wrapping scheme is agnostic to the credential’s signature algorithm, PQ-capable passkeys (e.g. ML-DSA-based, as they appear in authenticators) can be adopted without any change to the record format; a future `prf-v2` scheme identifier is reserved for the case where the KDF or cipher itself must move.

## 8 Evaluation and Discussion

The reference implementation is evaluated along three axes. *Security*: the zero-knowledge argument of Sect. 4.3 covers the persisted state; the unit-test suite additionally pins the negative behaviors the argument relies on — wrong-key decryption is indistinguishable from corruption, malformed records never reach the cipher, and non-increasing signature counters abort the unlock. *Performance*: unlock cost is dominated by the WebAuthn ceremony itself (user verification takes seconds), against which one HKDF derivation and one AES-GCM decryption of a 32-byte payload are negligible; on the password fallback path, scrypt with  $N=2^{17}$  deliberately costs a few hundred milliseconds on commodity hardware, a floor chosen so that offline guessing, not interactive unlock, bears the cost. *Usability*: PRF-based unlock replaces master-password entry with a platform biometric gesture, but availability is uneven across browsers, platforms, and embedded WebViews; the tri-state capability detection and the `pw-v1` fallback exist precisely so that applications degrade gracefully rather than probe destructively. A quantitative cross-device benchmark of ceremony latency and PRF availability is left as future work.

## 9 Conclusion

This paper formalized a WebAuthn PRF-based vault key wrapping scheme — PRF output as HKDF input keying material under a frozen, versioned domain-separation label, wrapping a non-extractable AES-256-GCM vault key — together with an IndexedDB storage architecture whose entire persisted state

provably contains no recomputable key inputs. The scheme integrates client-side replay verification, a memory-hard password fallback for clients where PRF is unavailable, and a ceremony-free migration path from the predecessor TrustVault-PWA format. The open-source `webauthn-prf-zktv` library [7] packages these patterns behind small, typed entry points so that other web applications can adopt PRF-based zero-knowledge vaults without re-deriving the storage and key-hygiene discipline described here.

## References

1. hole.storage: client-side file encryption with the WebAuthn PRF extension. Open-source project, 2024. TODO-VERIFY: confirm project name and URL before submission.
2. Bitwarden. PRF WebAuthn and its role in passkeys. Bitwarden Blog, 2024.
3. Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, IETF, 2010.
4. Matthew Miller. The WebAuthn PRF extension. SimpleWebAuthn documentation, 2024.
5. Dustin Moody et al. Transition to post-quantum cryptography standards. NIST Internal Report 8547 (initial public draft), National Institute of Standards and Technology, 2024.
6. Ian Pinto. TrustVault-PWA: security architecture and threat model documentation. GitHub repository, 2026. TODO-VERIFY: confirm repository visibility and URL before submission.
7. Ian Pinto. `webauthn-prf-zktv`: WebAuthn PRF-backed vault key wrapping with zero-knowledge IndexedDB storage, v0.1.0. npm package and GitHub repository, 2026. Reference implementation for this paper; also published as `webauthn-prf-zktv` on npm.
8. W3C Web Authentication Working Group. Web authentication: An API for accessing public key credentials, level 3. W3C Working Draft, 2025. §10.1.4 defines the `prf` client extension.
9. Yubico. The WebAuthn PRF extension. Yubico Developers documentation, 2024.